# Gráfica Multimedia

## Producción

Lourdes Marco · lmarco@esdmadrid.es

ESDM

# JavaScript

## Testear y comprobar con la consola

# console.log()

Es un método o **función** que se usa para **mostrar un mensaje en la consola**.

La consola se usa mucho en programación web para **testear**, ir comprobando que el código es correcto y que devuelve lo que esperamos.

JS

```
console.log(“Hola mundo”);

let total = 3 * 345;
console.log(total);
```

ESDM

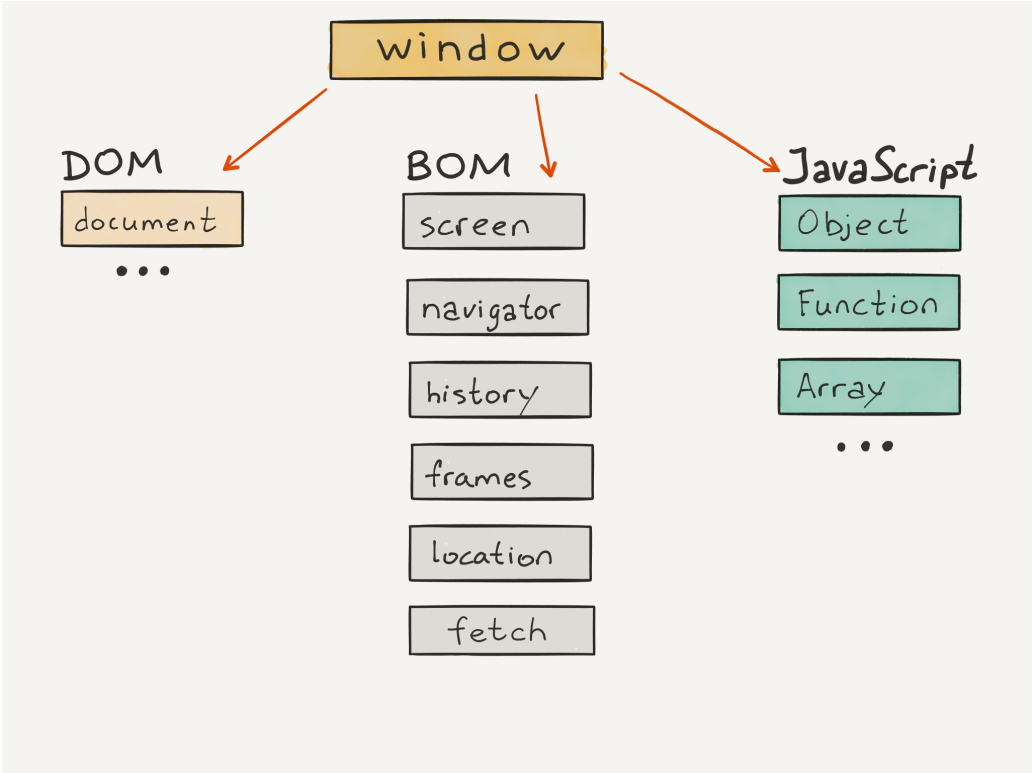
# JavaScript

## El objeto window y el BOM

# JavaScript

JS nos da acceso a mucha información del navegador a través de un **objeto global llamado window.**

Desde window podemos acceder al **DOM** y al **BOM**. El Browser Object Document (BOM) nos da acceso a objetos como screen (pantalla), history (historial de navegación), location (URL), etc… que nos permiten obtener información sobre el navegador.



ESDM

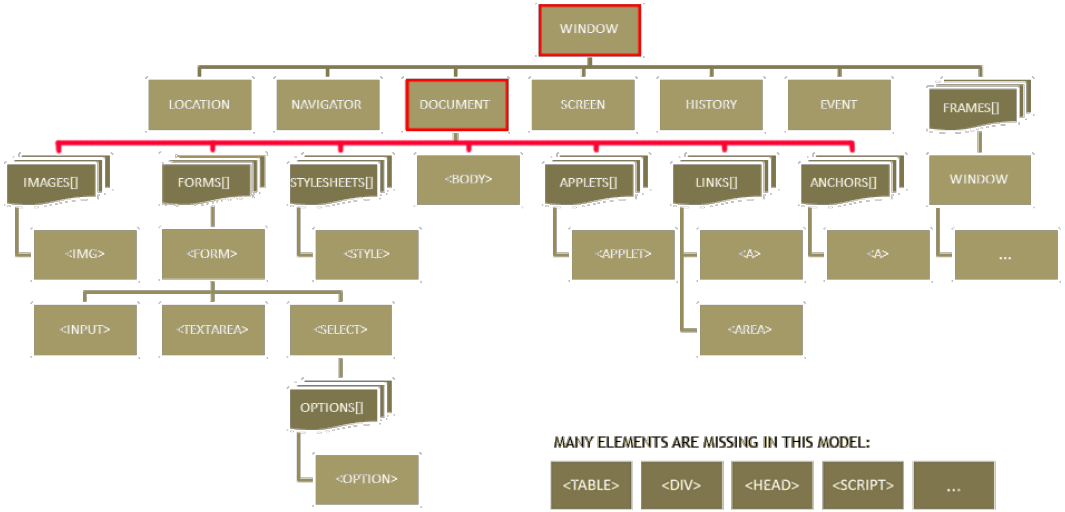
# JavaScript

## El DOM

# DOM

De forma similar a lo que sucede con el BOM, los navegadores transforman automáticamente las páginas HTML en una estructura más eficiente de manipular, que conocemos como **Document Object Model o DOM.**

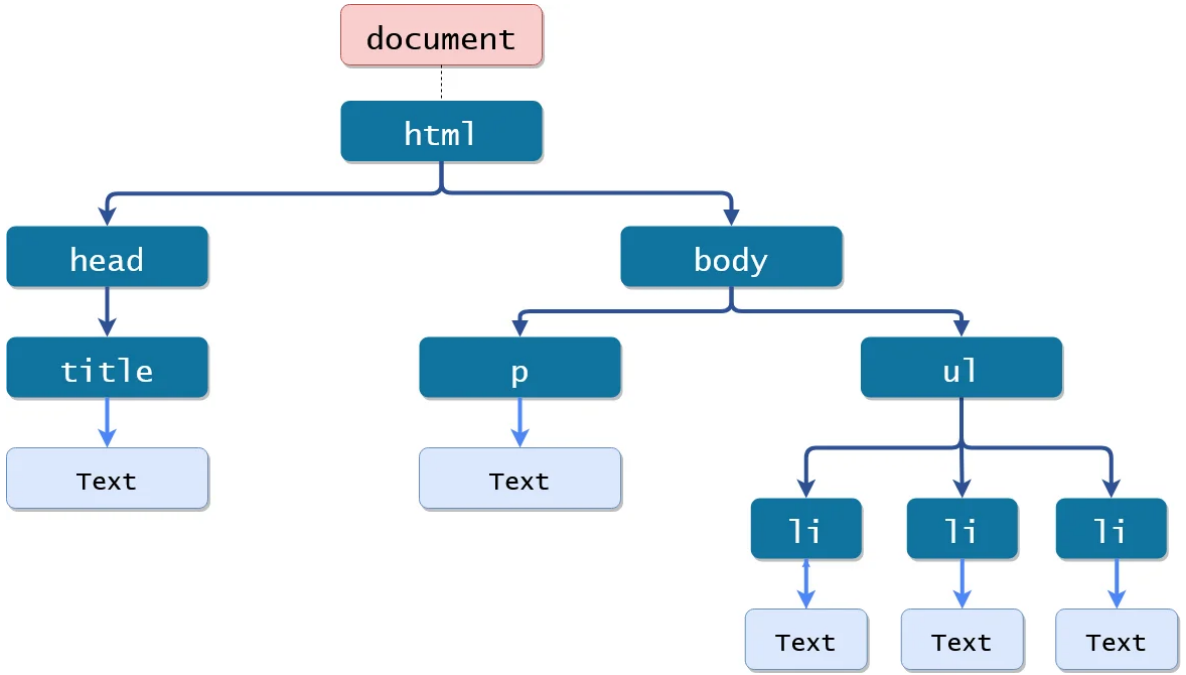
El DOM es un **conjunto de elementos llamados nodos**, que están interconectados. Por su aspecto, la unión de todos estos nodos se llama "**árbol de nodos**".



# El objeto document

A través del **objeto document,** JS proporciona una API que permite **añadir, eliminar y modificar elementos en el documento HTML**.

**document es el nodo raíz del DOM**, y tiene un hijo principal y único que es el elemento <html>. Cada marca (etiqueta) puede ser representada por un nodo en el árbol.



# Selectores JS

El objeto document tiene varios métodos que permiten **seleccionar uno o varios elementos (etiquetas)** del fichero HTML.

JS

```
document.getElementById(“boton1”);
document.getElementsByName(“nombre”);
document.getElementsByClassName(“rojo”);
document.getElementsByTagName(“h1”);

document.querySelector(“#boton1”);
document.querySelectorAll(“.clase”);
```

# Probando querySelector

El objeto document tiene varios métodos que permiten **seleccionar uno o varios elementos (etiquetas)** del fichero HTML.

HMTL

```
<p id="lorem"></p>
```

JS

```
let parrafo = document.querySelector("#lorem");
console.log(parrafo);
```

# Cambiar el texto o contenido HTML

Una vez seleccionado un elemento, podemos **cambiar su contenido textual** con **textContent,** pero también podemos **cambiar su contenido añadiendo texto y etiquetas HTML** con **innerHTML**.

HMTL

```
<p id="lorem"></p>
```

JS

```
let parrafo = document.querySelector("#lorem");
parrafo.innerHTML = "Lorem <em>Ipsum</em>";
parrafo.textContent = "Lorem Ipsum";
```

# JavaScript

## Eventos: click

# ¿Qué es un evento?

Los eventos son todas aquellas **acciones que suceden en el navegador,** y que éste es capaz de **detectar** para que podamos programar cuando suceden.

Los eventos **pueden ocurrir cuando el usuario interactúa**, por ejemplo, **cuando hace clic, doble clic, o pulsa una tecla.**
Pero también pueden ocurrir cuando se carga la página, o se redimensiona la ventana.



# Manejadores de eventos

Cada evento puede tener un manejador de eventos (también listener o escuchador de eventos), que es un bloque de código que se ejecutará cuando el evento ocurra.

Es decir, una cosa es que el usuario haga clic y otra distinta es que preparemos el código para "manejar" o "escuchar" este clic.

| Evento | Descripción | Manejador |
|---|---|---|
| **click** | Ocurre cuando el usuario hace clic sobre un elemento | **onclick** |
| **dblclick** | Ocurre cuando el usuario hace doble clic sobre un elemento | **ondblclick** |
| **load** | Ocurre cuando un objeto ha sido cargado | **onload** |
| **change** | Ocurre cuando el contenido de un elemento ha cambiado (input, select, textarea…) | **onchange** |
| **mouseover** | Ocurre cuando el puntero se mueve dentro de un elemento | **onmouseover** |
| **mouseout** | Ocurre cuando el puntero se mueve fuera de un elemento | **onmouseout** |
| **mousedown** | Ocurre cuando el usuario hace clic sobre un elemento | **onmousedown** |
| **mouseup** | Ocurre cuando el usuario suelta el clic sobre un elemento | **onmouseup** |
| **blur** | Ocurre cuando un elemento pierde el foco | **onblur** |
| **focus** | Ocurre cuando un elemento obtiene el foco | **onfocus** |
| **keydown** | Ocurre cuando el usuario presiona una tecla | **onkeydown** |

ESDM

# Formas de escuchar

Podemos usar los manejadores / listeners de distintas formas, cada una con peculiaridades distintas.

- Como **atributo en una etiqueta HTML**
- Como **propiedad** en un elemento seleccionado con JS
- Con en **método addEventListener()**



# Como atributo HTML

HTML

```
<!-- código JS en HTML -->
<button onclick=”alert(‘Mi primer evento JS’);”>
        Ver Alerta
</button>
```

# Como propiedad de un elemento seleccionado

HTML

```
<button>Alertar</button>
```

JS

```
let button = document.querySelector("button");
button.onclick = () => alert("Mi segundo evento JS");
```

# Evento addEventListener

El método addEventListener() permite registrar un “escuchador” de eventos a un elemento seleccionado con JS.

A diferencia de las dos formas anteriores, nos permite añadir varios “escuchadores” a un mismo elemento, aunque sean del mismo tipo. (Ej. dos eventos click)

```
elem.addEventListener(event, function, [capture]);
```

- El primer parámetro es el **tipo de evento (**click, mousedown...)
- El segundo es **la función** que queremos ejecutar
- El tercer **parámetro es opcional.** Se trata de un valor booleano para especificar si se utiliza bubbling (false) o capturing (true).

# Con el método addEventListener()

HTML

```
<button>Alertar</button>
<p></p>
```

JS

```
const p = document.querySelector("p");
document.querySelector("button").addEventListener("click", () =>
{
  p.textContent += "hola mundo ";
});
```